



Day 51



“CLOUD SECURITY”

AWS Specific Penetration Testing:

Attempt to Identify S3 Buckets:

Manual Methods:

- HTML inspection, brute-force, Google Dork, DNS caching, Bing reverse IP, GitHub, CDN object URL, Burp Suite.

Automated Tools:

- Bucket Finder, S3 Inspector, S3Scanner, Lazy S3, S3 Bucket Finder.

Check for S3 Bucket Permissions:

1. Check ACLs at bucket or object level to control access.
2. AWS CLI commands for testing:
 - READ: `aws s3 ls s3://[bucketname] --no-sign-request`
 - WRITE: `aws s3 cp localfile s3://[bucketname]/test.txt --no-sign-request`
 - READ_ACP: `aws s3api get-bucket-acl --bucket [bucketname] --no-sign`
 - WRITE_ACP: `aws s3api put-bucket-acl --bucket [bucketname] --no-sign-request (test ACL changes)`

Attempt to Create New Policy Version:

1. Test if AWS administrator access can be obtained by creating new policy versions.
2. Use the command:

```
aws iam create-policy-version
--policy-arn target_policy_arn
--policy-document [policy-document-name/path].json
--set-as-default
```

Attempt to Set an Existing Policy Version as Default:

1. Test risks of inactive policy versions by setting one as default.
2. Use the command:

```
aws iam set-default-policy-version
--policy-arn target_policy_arn
--version-id [new version-id]
```

Attempt to Obtain Access to EC2 Instance/Role Permissions:

1. Test access to EC2 instance/service role permissions of an AWS account.
2. Create an EC2 instance using an existing instance profile with iam:PassRole and ec2:RunInstances permissions.
3. Login to the EC2 instance and retrieve AWS keys from instance metadata to access permissions.
4. Use or import an SSH key for instance access:

```
aws ec2 run-instances  
  
-image-id [image-id]  
  
-instance-type [instance-type]  
  
-iam-instance-profile Name=[iam-instance-profile Name]  
  
-key-name [key-name]  
  
-security-group-ids [security-group-ids]
```

Attempt to Create a New User Access Key:

1. Test if privilege escalation to AWS administrator is possible.
2. Attempt to create a new access key for a normal user:

```
aws iam create-access-key -user-name target_user
```

Attempt to Create a New Login Profile:

1. Test if privilege escalation to AWS administrator is possible by creating a login profile with a normal user account.
2. Create a JSON skeleton for the login profile:

```
aws iam create-login-profile --generate-cli-skeleton > create-login-profile.json
```
3. Use the JSON file to create a new login profile for the IAM user:

```
aws iam create-login-profile --cli-input-json file://create-login-profile.json
```
4. Verify that normal users cannot create login profiles that grant excessive privileges.

Attempt to Update an Existing Login Profile

1. Test if a normal user can **escalate privileges** by updating a login profile.
2. Command example:

```
aws iam update-login-profile --user-name John --password <password>
```
3. Verify that regular users **cannot modify login profiles** to gain admin access.

Attempt to Attach a Policy to a User

1. Test privilege escalation by attaching an AWS managed policy to an IAM user.
2. Command example:

```
aws iam attach-user-policy --policy-arn arn:aws:iam:ACCOUNT-ID:aws:policy/AdministratorAccess --user-name John
```
3. Ensure normal users cannot attach admin policies to themselves.

Attempt to Attach a Policy to a Group

1. Test privilege escalation by attaching an **AWS managed policy** to an IAM group.
2. Command example:

```
aws iam attach-group-policy --policy-arn arn:aws:iam::aws:policy/ReadOnlyAccess --group-name Accounts
```

Attempt to Attach a Policy to a Role

1. Test privilege escalation by attaching an AWS managed policy to an IAM role.
2. Command example:

```
aws iam attach-role-policy --policy-arn arn:aws:iam::aws:policy/ReadOnlyAccess --role-name ReadOnlyRole
```

Attempt to Create/Update an Inline Policy for a User

1. Test privilege escalation by creating or updating an inline policy for an IAM user.
2. Command example:

```
aws iam put-user-policy --user-name Bob --policy-name ExamplePolicy --policy-document file://AdminPolicy.json
```

Attempt to Create/Update an Inline Policy for a Group

1. Test privilege escalation by creating or updating an inline policy for an IAM group.
2. Command example:

```
aws iam put-group-policy --group-name Administrator --policy-document file://AdminPolicy.json --pol
```

3. Purpose: Verify if group members can gain unauthorized permissions on resources through inline policies.

Attempt to Create/Update an Inline Policy for a Role

1. Test privilege escalation by creating or updating an **inline policy** for an IAM role.
2. Command example:

```
aws iam put-role-policy --role-name RoleTest --policy-name ExamplePolicy --policy-document file://AdminPolicy.json
```
3. Purpose: Verify if the role can gain unauthorized permissions on cloud resources through inline policies.

Attempt to Add a User to a Group

1. Test privilege escalation by adding a user to an IAM group.
2. Command example:

```
aws iam add-user-to-group --user-name John --group-name Administrators
```
3. Purpose: Check if the user can gain the permissions of the IAM group without proper authorization.

Attempt to Update AssumeRolePolicyDocument of a Role

1. Test if a user can modify the trust policy of an IAM role to assume that role and gain elevated privileges.
2. Command example:

```
aws iam update-assume-role-policy --role-name RoleTest --policy-document file:///RoleTest-Trust-Policy.json
```
3. Purpose: Check whether unauthorized users can change role trust relationships to escalate permissions.

--The End--